

Implementación de Búsquedas y Filtros

Actividad N°: 22 **Fecha:** 30/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

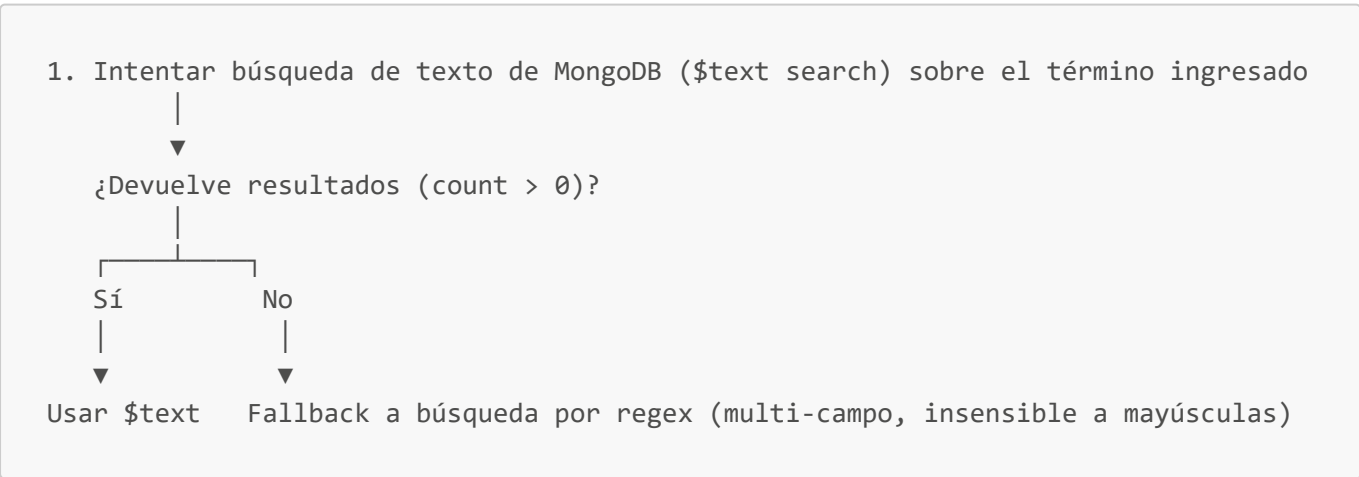
Implementar el motor de búsqueda y los filtros de la pantalla de tickets, cubriendo el criterio de aceptación de HU-02 (búsqueda por nombre, cédula, email o Ticket ID).

2. Parámetros de búsqueda soportados

Parámetro	Campo(s) afectado(s)	Tipo de coincidencia
search	First Name, Last Name, Email, Ticket ID, Transaction ID, Numero de Cedula	Texto general, insensible a mayúsculas (regex i)
ticketIdSearch	Ticket ID	Coincidencia exacta (string)
seatSearch	Seat	Texto parcial, insensible a mayúsculas
puntoTrabajo	puntoTrabajo	Exacto (solo aplicable/forzado para Jefe consultando; en Staff se ignora y se usa el propio)
sortBy / sortOrder	Cualquier campo ordenable	Ascendente/descendente

3. Estrategia de búsqueda en `getTicketsByPuntoVenta`

Se implementó una estrategia de dos niveles para optimizar el rendimiento en colecciones grandes:



Esto permite aprovechar índices de texto cuando existen y siguen siendo efectivos, sin dejar de funcionar si la búsqueda de texto no encuentra coincidencias (por ejemplo, búsquedas parciales de cédula que `$text` no maneja bien por defecto).

4. Combinación de filtros

Los filtros se combinan con operadores `$and/$or` de MongoDB según la cantidad de criterios activos: un único filtro se aplica directo; dos o más se combinan siempre con `$and` para que actúen como intersección (por ejemplo, "búsqueda general" y "localidad específica" al mismo tiempo), evitando el error común de que agregar un segundo filtro amplíe accidentalmente los resultados en vez de acotarlos.

5. Corrección aplicada: campo de cédula duplicado/muerto

Se identificó que las condiciones de búsqueda por cédula incluían dos nombres de campo distintos:

```
{ 'Numero de Cedula:': searchRegex },      // Coincide con el schema real
(Ticket.js)
{ 'Número de Cédula: ': searchRegex }      // Con tilde y espacio final – nunca
coincide con ningún documento
```

El segundo nunca puede producir coincidencias porque no existe ningún campo con ese nombre exacto en el esquema (`Ticket.js` define `'Numero de Cedula:'`, sin tilde). Era código muerto inofensivo (no generaba errores ni resultados incorrectos), pero agregaba ruido y confusión para quien mantenga el código a futuro.

Archivos corregidos: `backend/src/controllers/ticketController.js` y `backend/src/controllers/puntoVentaController.js` (3 ocurrencias en total: `getTickets`, `getTicketsByPuntoVenta`, `getTicketsForStaff`).

```
- { 'Numero de Cedula:': searchRegex },
- { 'Número de Cédula: ': searchRegex }
+ { 'Numero de Cedula:': searchRegex }
```

6. Frontend — filtros en `TicketsPage.jsx`

- Campo de búsqueda general con debounce para no disparar una petición por cada tecla presionada.
- Campo específico de búsqueda por localidad (`seatSearch`).
- Selector de punto de venta (solo visible para Jefe; el Staff no lo ve porque su punto de trabajo ya está fijo).
- Indicador de "usuario interactuando" para pausar actualizaciones en tiempo real mientras se escribe una búsqueda (ver Semana 1, RF-13).

7. Conclusiones del día

El motor de búsqueda queda implementado con una estrategia de dos niveles (texto + regex de respaldo) y se corrigió una inconsistencia de nombres de campo que, aunque no afectaba el resultado funcional, representaba deuda técnica innecesaria.

Observaciones: Corrección de campo duplicado aplicada y verificada; sin observaciones adicionales.